

Chapter-6
Model Selection

Chapter 6: Model Selection and Evaluation

- This chapter delves into the critical process of selecting the best-performing machine learning model for a given task and rigorously evaluating its effectiveness.
- It covers techniques for tuning model parameters, optimizing performance, managing model complexity, understanding common pitfalls like overfitting and underfitting, and employing robust evaluation methods.

Model Selection

❖The process of choosing the most appropriate model or algorithm for a given machine learning problems.

❖This decision depends on:

- ☑ nature of the data
- ☑ problem type (e.g., classification, regression),
- ☑ accuracy requirements
- ☑ interpretability
- ☑ computational cost/efficiency
- ☑ model complexity (e.g., linear vs. non-linear models)
- ☑ domain-specific requirements

Hyperparameter Tuning

Parameters

Model Parameters

These are the parameters of the model that can be determined by training with training data. These can be considered as internal Parameters.

- **Weights**
- **Bias**

$$Y = w*X + b$$

Hyperparameters

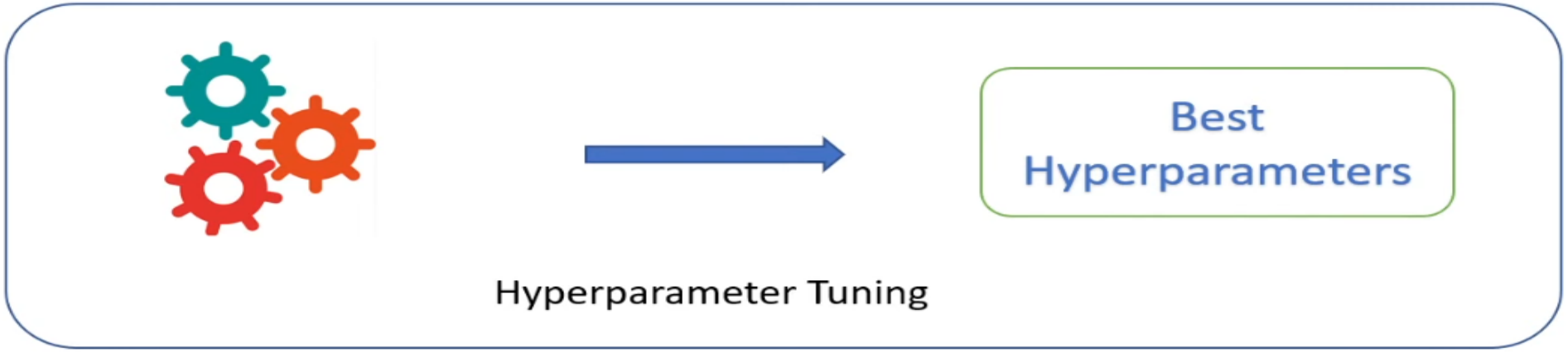
Hyperparameters are parameters whose values control the learning process. These are adjustable parameters used to obtain an optimal model. External Parameters.

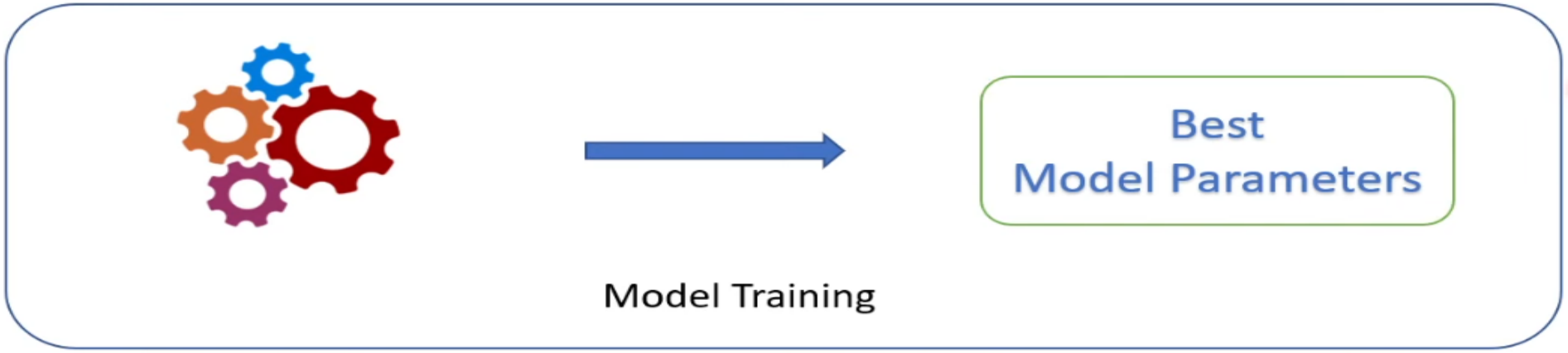
- **Learning rate**
- **Number of Epochs**
- **n_estimators**

Hyperparameter Tuning

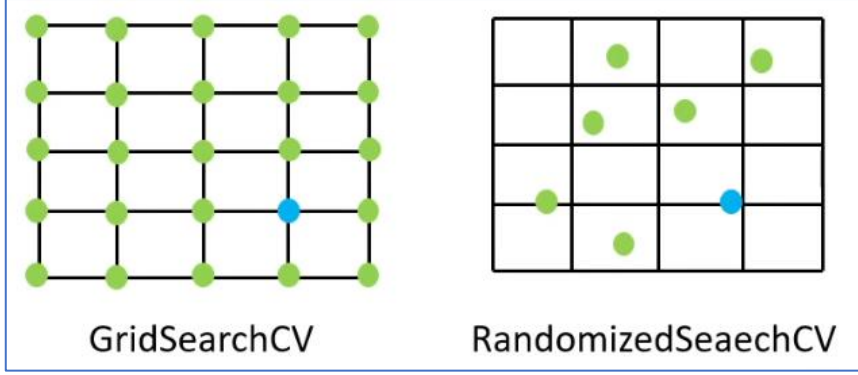
- Machine learning models have parameters learned from the data during training (e.g., the weights in a neural network). They also have hyperparameters, which are set *before* training begins (e.g., learning rate in gradient descent, the number of layers in a neural network, the regularization strength, depth of trees).
- *Hyperparameter tuning is the process of finding the optimal set of hyperparameters that results in the best model performance.*
- Configurable settings that are **not learned** from data(during training).
- Adjusting hyperparameters maximize the model's performance.

Hyperparameter Tuning





Techniques:



1. Grid Search:
   - ❖Exhaustively tries all possible combinations of hyperparameter values specified in a grid.
   - ❖The model is trained and evaluated for every possible combination of hyperparameter values in the grid.
   - ❖Computationally expensive for large hyperparameter spaces, but effective for small search spaces.
2. Random Search:
   - ❖Randomly selects combinations, often faster than Grid Search.
   - ❖Often faster and more efficient for large search spaces.
3. Bayesian Optimization:
   - ❖ Uses probabilistic models to find optimal parameters efficiently.

6.2 Optimize the Performance of the Model

- This is a crucial, iterative process aimed at improving your model's ability to generalize and make accurate predictions on new, unseen data.
- Optimizing a model’s performance means making it as accurate and reliable as possible.
- Its main goal is to minimize the generalization error (error on unseen data) according to the chosen performance metric(s) (e.g., accuracy, F1-score, RMSE, AUC).

Strategies for Optimization:

1. Feature Engineering: Improve model input quality.
   - ✅**Feature Extraction:** Creating new features from existing ones.
   - ✅**Feature Creation:** Designing and training new features for specific tasks.
   - ✅**Feature Selection:** Choosing the most relevant features to use in a model.
   - ✅**Feature Transformation:** Modifying existing features to improve their suitability for a model.
2. Data Preprocessing:
   - ☑ Normalize or standardize features.
   - ☑ Handle missing values and outliers effectively.

Cont'd

3. Algorithm Selection: Choose algorithms suited to the problem.
   - For instance: Decision trees for categorical data. Neural networks for high-dimensional data.
4. Hyperparameter Tuning: Find the optimal settings for your model.
   - Grid Search (test all combinations)
   - Random Search (test random combinations)
5. Regularization
   - Add penalties to large model parameters to avoid overfitting.
     - L1 (Lasso) — helps with feature selection.
     - L2 (Ridge) — shrinks coefficients to prevent overfitting(penalizes large weight).

Cont'd

6. Cross-Validation
   - Evaluate models more reliably by splitting data into multiple parts.
   - Ensures model performs consistently across different datasets.
7. Ensemble Methods
   - Combine multiple models to improve predictions/performance.
   - Helps reduce both bias and variance.
     - Bagging: Reduce variance by averaging predictions (e.g., RandomForest, BaggingClassifier).
     - Boosting: Reduce bias by iteratively correcting errors (e.g., XGBoost, LightGBM, AdaBoost).
     - Stacking: Combine predictions of multiple models using a meta-learner.

6.3 Control Model Complexity

Why Control Complexity?

❖To ensure the model generalizes well to unseen data.

❖Avoid overfitting (too complex) or underfitting (too simple).

- Too Simple (Low Complexity): Leads to underfitting - the model cannot capture the underlying patterns in the data.
- Too Complex (High Complexity): Leads to overfitting - the model learns noise and specific details of the training data, failing to generalize.

Methods to Control Complexity:

❖Regularization: Add penalty terms to the loss function to discourage overly complex models.

- Examples:
  - L1 Regularization (Lasso): Encourages sparsity.
  - L2 Regularization (Ridge): Penalizes large weights.

❖Dimensionality Reduction: Reducing the number of input features (e.g., using Principal Component Analysis - PCA, or feature selection techniques).

Methods to Control Complexity:

- ❖Pruning: Remove non-critical branches in decision trees.
- ❖Dropout (for Neural Networks):
  - ✓Randomly deactivate neurons during training to prevent over-reliance on specific features.
- ❖Restrict Depth/Size:
  - ✓Limit the number of layers, nodes, or splits in the model.

6.4 Overfitting and Underfitting

Overfitting

❖Overfitting (High Variance): The model learns the training data *too well*, including noise, patterns specific to the training data and random fluctuations. It performs very well on the training set but poorly on the unseen test set. The model does not generalize.

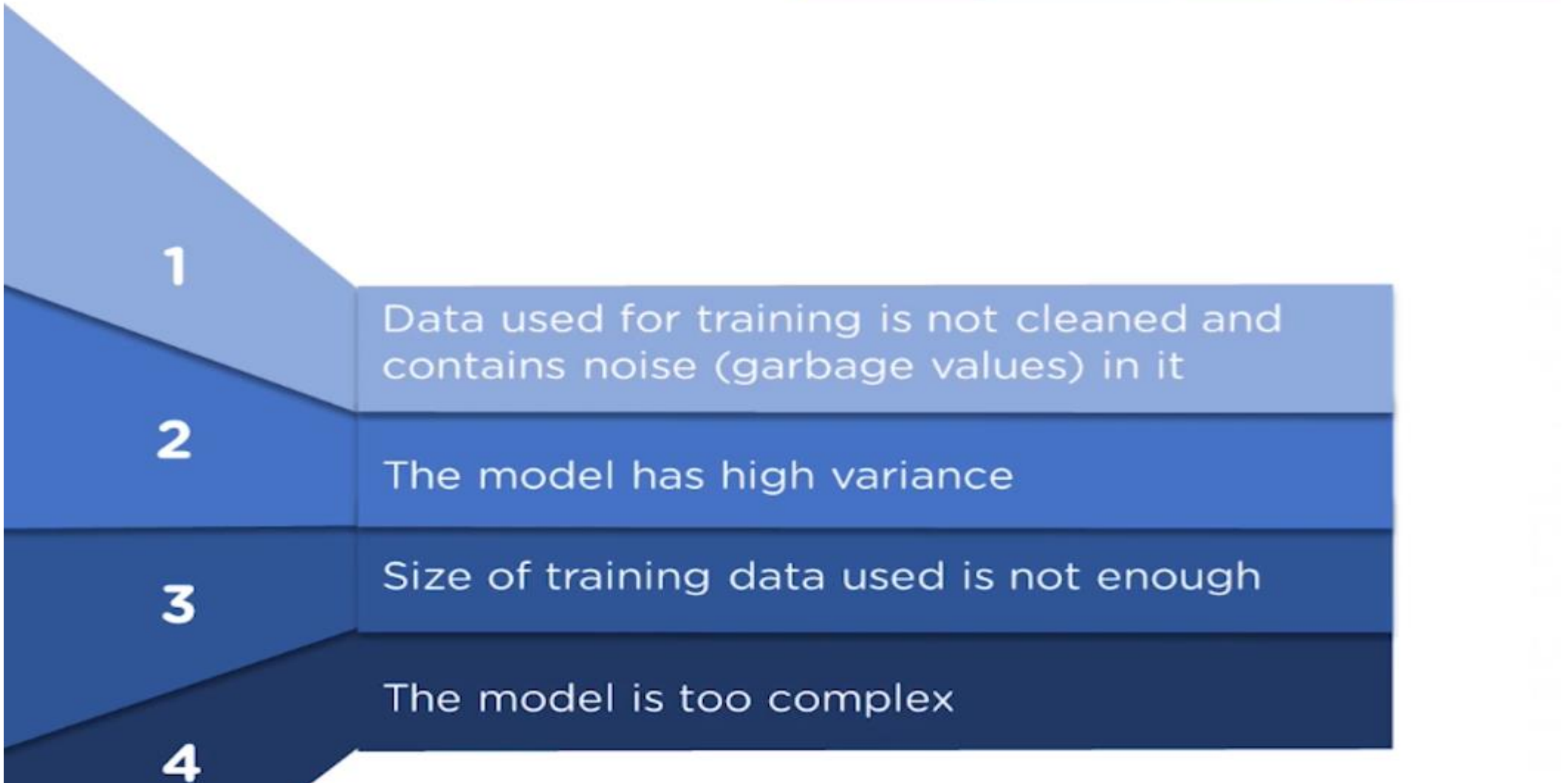
❖Symptoms:

- High training accuracy, low validation accuracy.
- Large difference between training and testing errors.

❖Solutions:

- Simplify the model (reduce complexity).
- Use regularization techniques.
- Increase the size of the training dataset.

Reasons for overfit

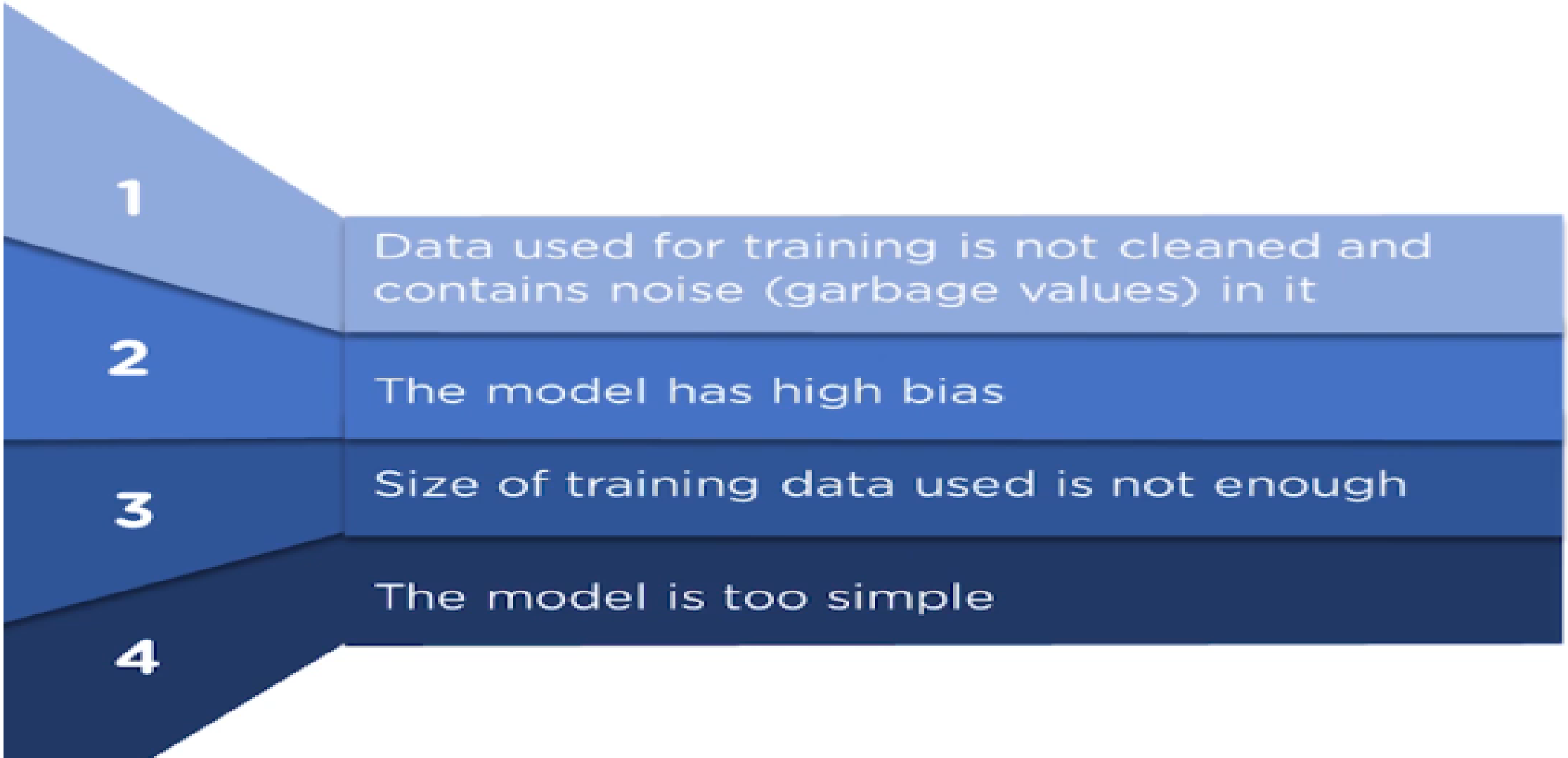


Cont'd

Underfitting

- Underfitting (High Bias): The model is too simple to capture the underlying structure of the data. It performs poorly on *both* the training set and the test set. The model has not learned enough.
- Symptoms:
  - Low training and validation accuracy.
  - Small difference between training and testing errors.
- Solutions:
  - Increase model complexity.
  - Use feature engineering to provide more informative inputs.
  - Train for more epochs.

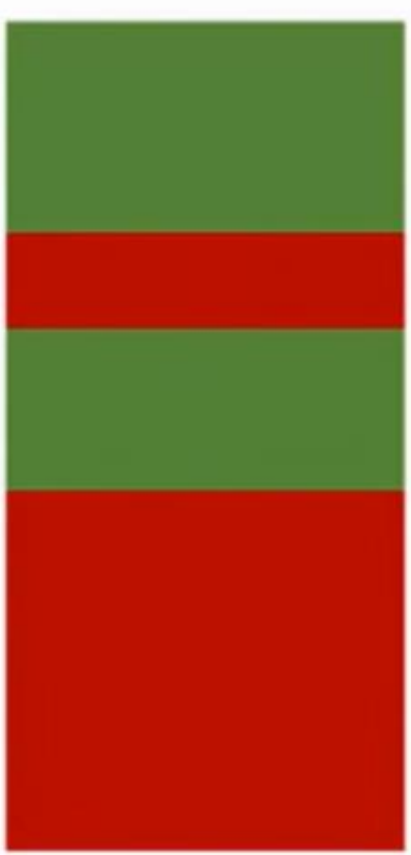
Reasons for Underfitting



6.5 Cross-Validation (CV)

- A technique to evaluate model performance by partitioning the data into training and validation sets multiple times.
- Techniques used to asses how well machine learning algorithm generalizes to unseen data
- Types:
  1. k-Fold Cross-Validation:
     - Split data into *k* subsets/parts (folds), Train the model on *k-1* folds and validate on the remaining one.
     - Model is trained k times and each time the model is trained on k-1 dataset remaining for testing.
  2. Leave-One-Out Cross-Validation (LOOCV):
     - Extreme case of K-fold where *k = n* (n = number of samples). Each data point is used once as a validation set.
     - Computationally expensive but useful for small datasets.
  3. Stratified k-Fold:
     - Ensures class distribution is consistent across folds (crucial for imbalanced datasets).
     - Ensures proportional representation of target classes in each fold.

Method 1: Split dataset into training and test set



70



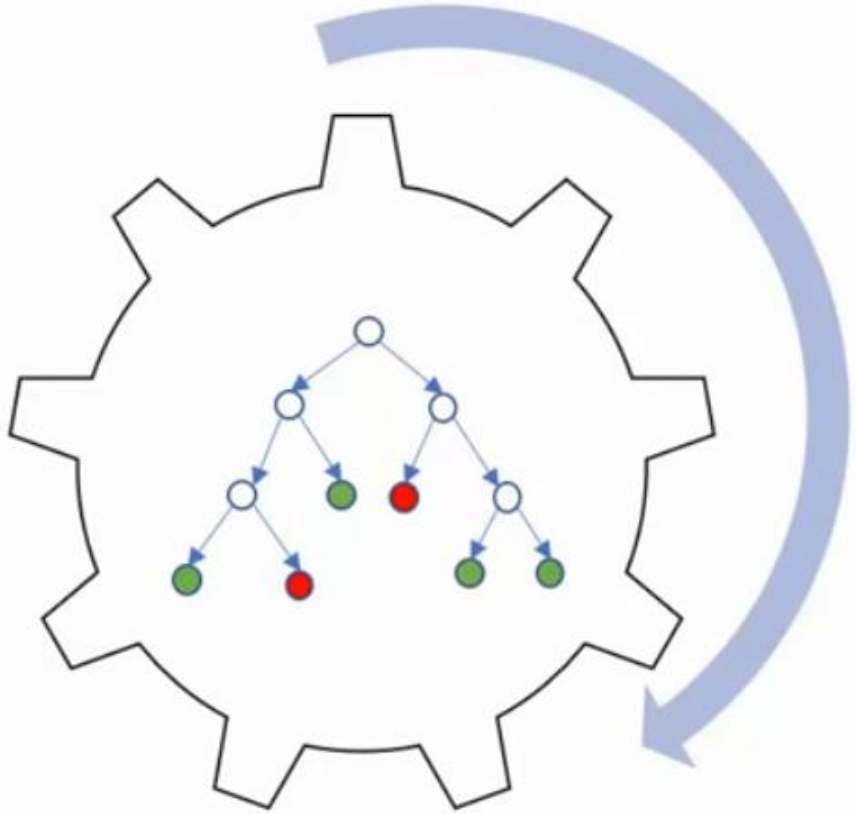
Train



Test

30





100 samples

Example



Train



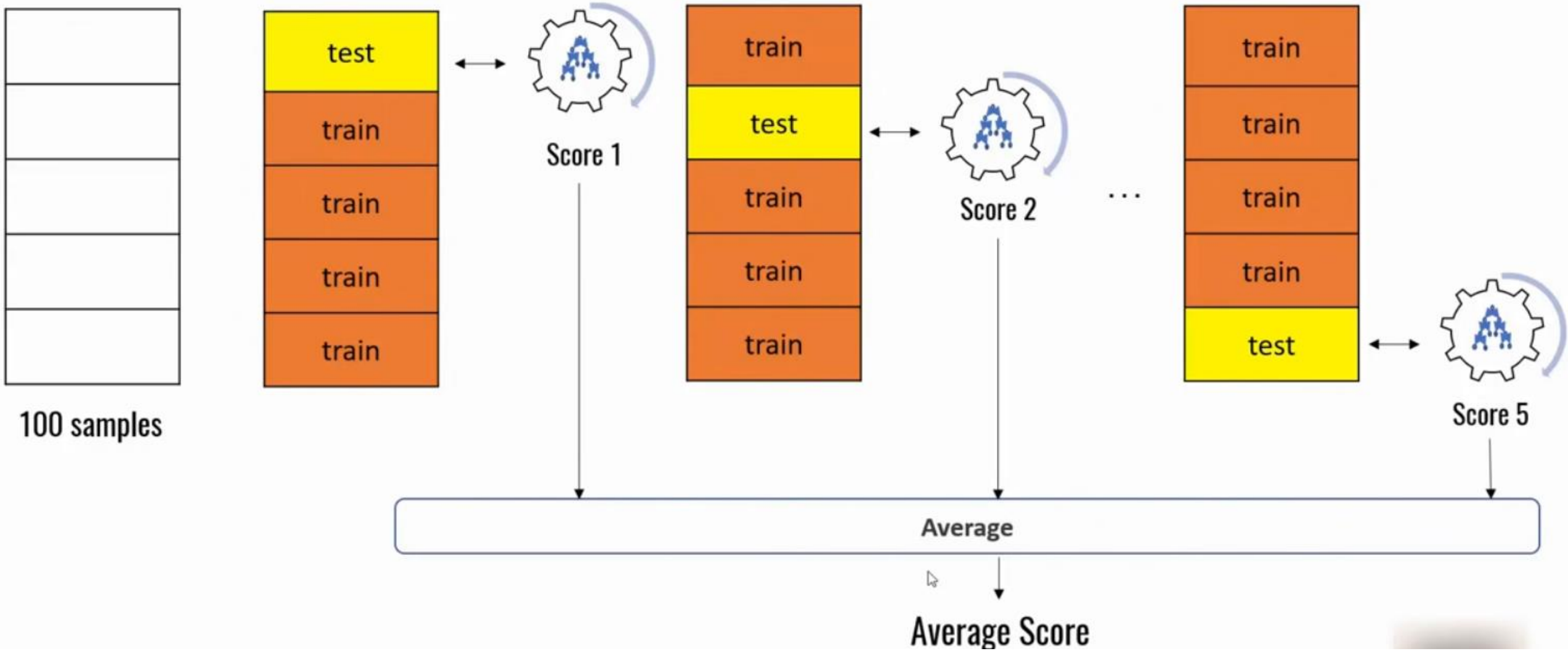
Test



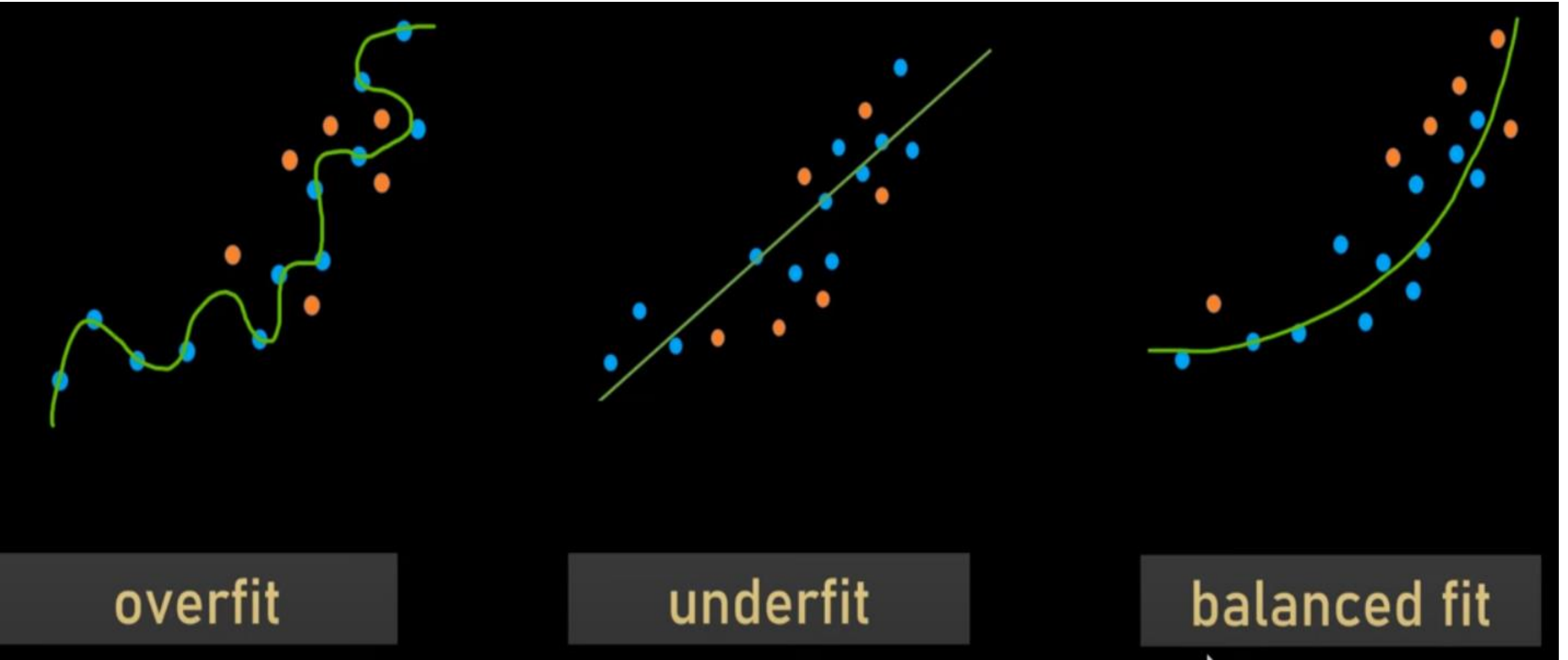
Remaining 30 questions

70 math questions

Method 2: Cross-Validation



Bias and Variance



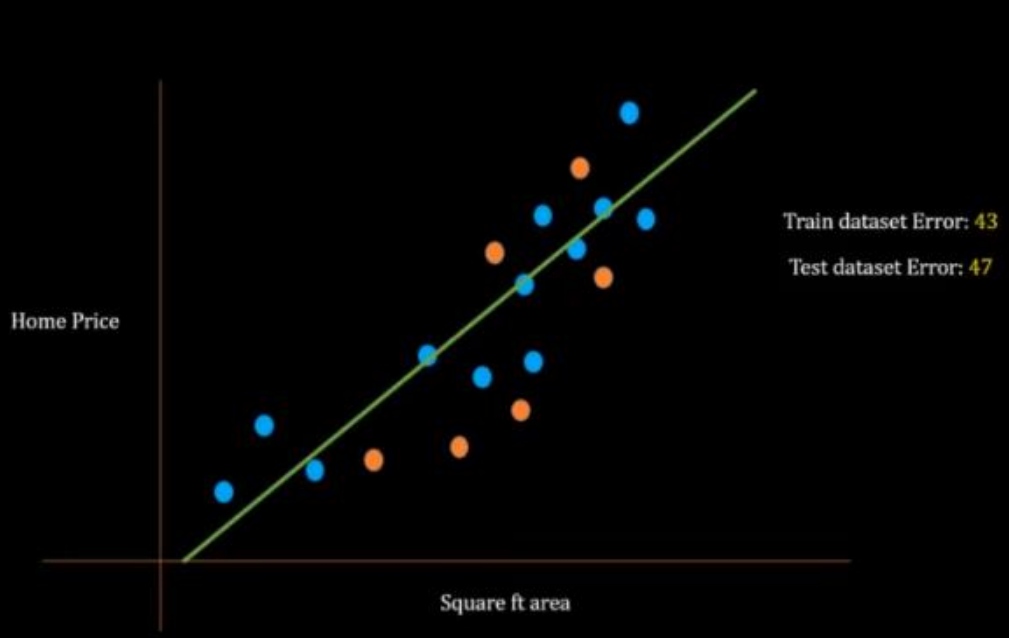
Bias

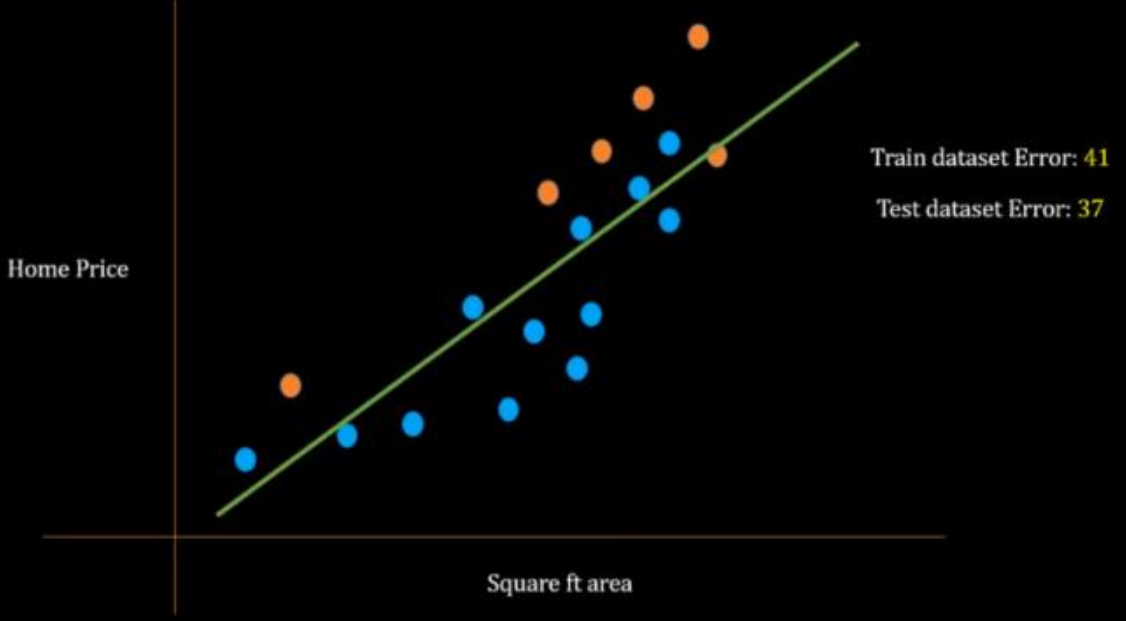
- Error due to overly simplistic model or oversimplify data.
- High bias leads to underfitting, where the model fails to capture underlying patterns in the data.
- Models with high bias have poor performance on both the training and testing datasets.
  - High training and validation/test error.

Bias

❖Bias can be caused by:

- Using linear models for non-linear problems.
    - E.g. Predicting house prices with a simple linear model (straight line) even if the real data has a complex pattern(non-linear relationship).
- Insufficient model complexity.





Test error = 47

Train error = 43

low variance

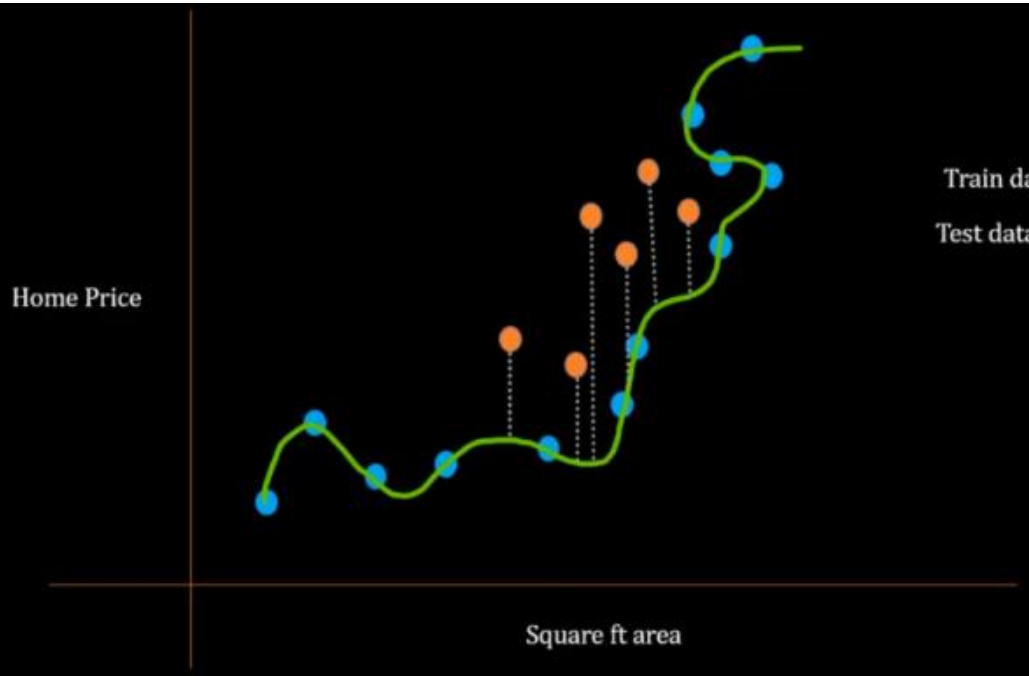
Test error = 37

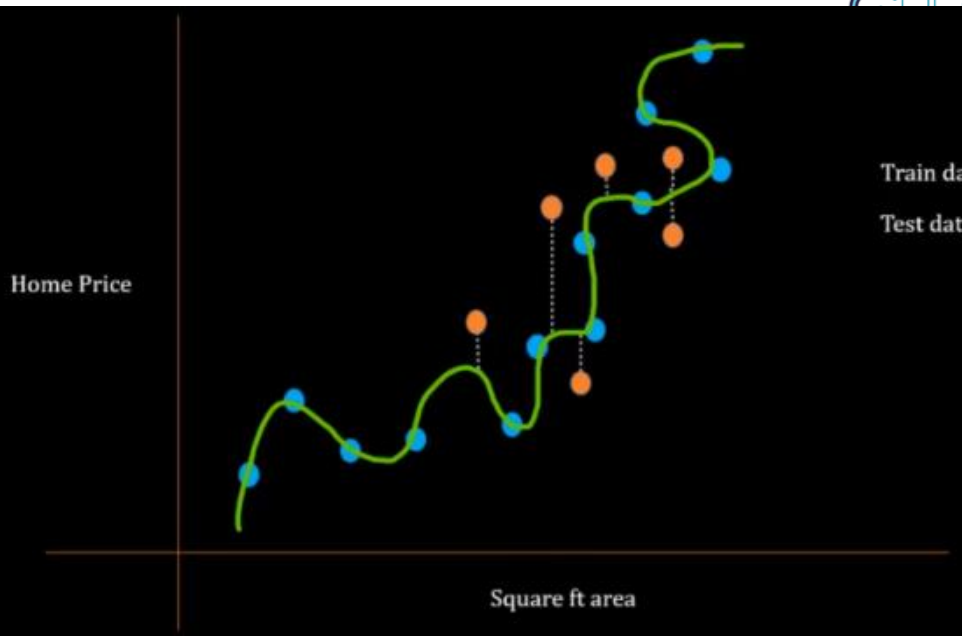
Train error = 41

Bias is a measurement of how accurately a model can capture a pattern in a training dataset. In above case since train error is big it is said to have a high bias.

Variance

- Error due to overly complex models((e.g., high-degree polynomial regression).
- Variation in prediction(inconsistent prediction by different model using different dataset)/(Sensitivity of the model to the training data. More sensitivity higher variance).
- If prediction is the same by different model it has low variance
- Model memorizes the noise in data
- High variance indicates that the model is too complex, leading to overfitting.
- Models with high variance perform well on the training data but poorly on unseen or test data.
  - Low training error but high validation/test error.
- Example: A deep decision tree that perfectly classifies the training data but performs badly on test data.

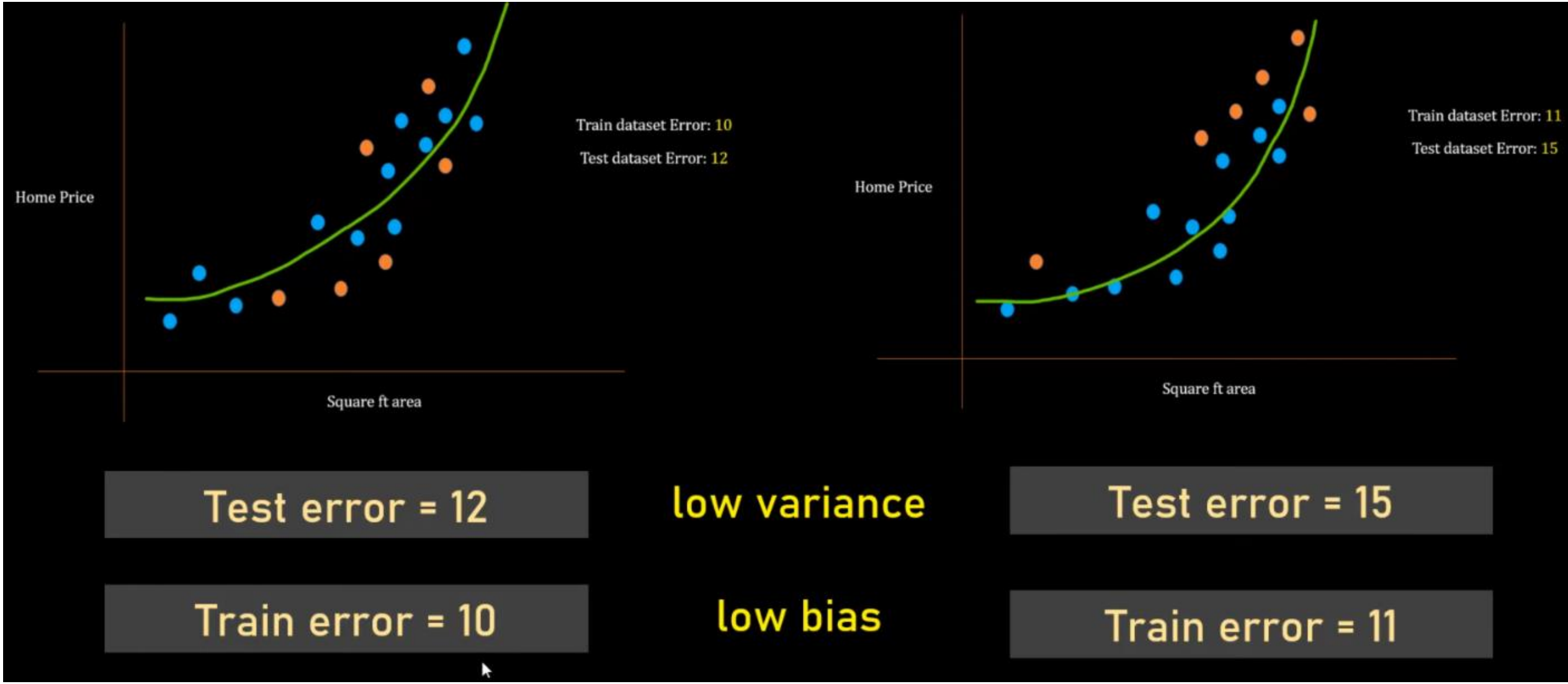




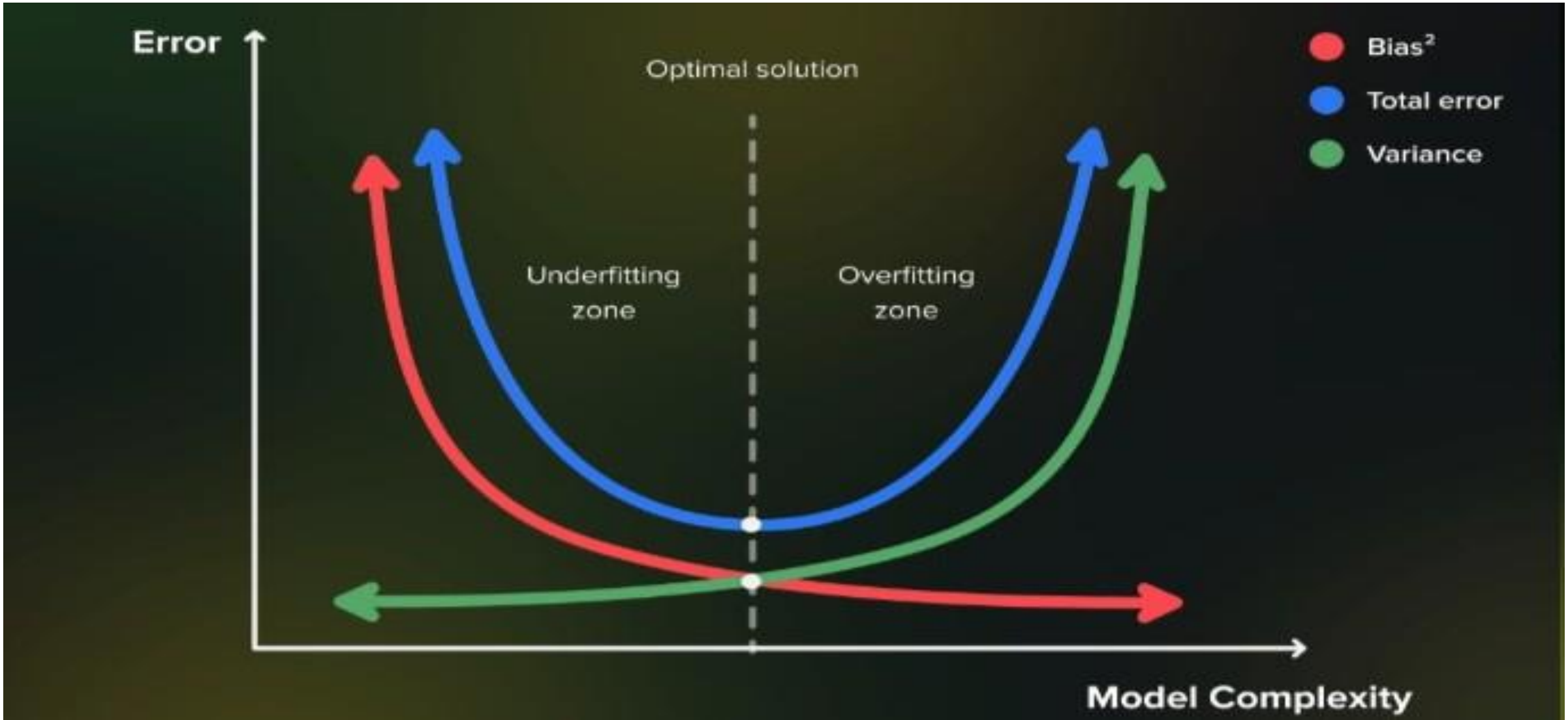
Test error = 100 ← high variance → Test error = 27

This is the case of high variance. Here test error varies greatly based on the selection of the training dataset.

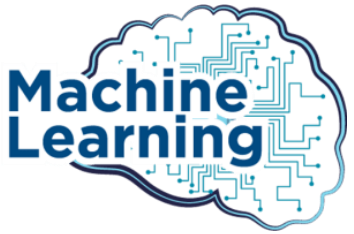
Balanced Fit Model



The Bias-Variance Trade-off



The Bias-Variance Trade-off



- The bias-variance trade-off is the balance between underfitting and overfitting.
- Balancing bias and variance is key to building robust machine learning models that generalize well to unseen data.
- The goal of bias variance tradeoff is to find the optimal level of model complexity that minimizes both bias and variance, resulting in the lowest total error.
- Tradeoff:
  - To achieve optimal performance, balance/minimize both bias and variance using techniques like regularization, cross-validation, and tuning.

Strategies to Address Bias and Variance

Ways to get balanced fitting model

❖ High Bias (Underfitting):

- Increase the model complexity(e.g. increase depth of decision tree, increase polynomials).
- Reduce regularization.
- Add more features or transformations.

❖ High Variance (Overfitting):

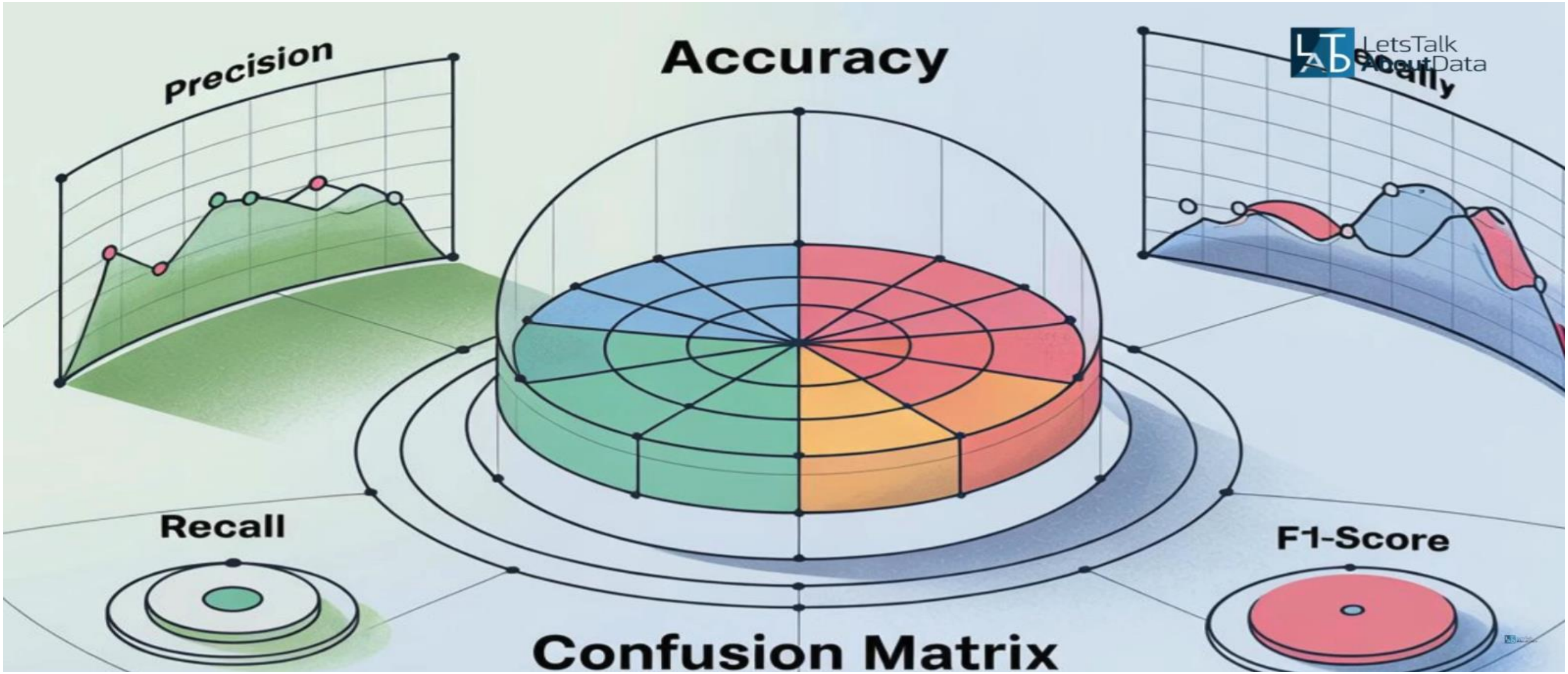
- Simplify the model by reducing its complexity.
- Cross validation
- Use regularization (L1, L2, or dropout).
- Increase the size of the training dataset.
- Use ensemble methods like bagging or boosting.

6.7 Performance Evaluation Methods

Metrics for Regression:

1. Mean Absolute Error (MAE):
   - ❖Measures average absolute difference between predicted and actual values.
2. Mean Squared Error (MSE):
   - ❖Penalizes larger errors more heavily than MAE.
3. R^2 Score:
   - ❖Indicates the proportion of variance explained by the model.

Metrics for Classification:



1. Classification Metrics

❖Used when the output is categorical (e.g., spam/not spam, disease/no disease).

1. Accuracy: Measures the percentage of correct predictions(proportion of correctly classified instances).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Use Case: Best used when classes are balanced.
- *Limitation*: Misleading for imbalanced datasets (e.g., 90% class A vs. 10% class B).

Cont'd

2. Precision: Measures how many predicted positives are actually positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Focus: Minimize false positives.

3. Recall: Measures how many actual positives are correctly predicted/identified

$$\text{Recall} = \frac{TP}{TP + FN}$$

Cont'd

4. F1 Score: Harmonic mean of precision and recall.

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- *Balances precision and recall*. Ideal for imbalanced data.

5. Confusion Matrix

❖Summarizes TP, TN, FP, FN for granular error analysis.

❖Helps to visualize the performance of a classification model.

Cont'd

- **ROC-AUC**:
  - ❖**ROC Curve**: Plots True Positive Rate (TPR) vs. False Positive Rate (FPR).
  - ❖**AUC**: Area Under the ROC Curve. **It m**easures the ability to distinguish between classes.

    Interpretation: AUC = 1 (perfect classifier), AUC = 0.5 (random guessing).

    Use Case: Compare models across different classification thresholds.

2. Regression Metrics

❖Used when the output is continuous (e.g., price prediction, temperature forecasting).

A. Mean Absolute Error (MAE): Average of absolute differences between predicted and actual values.

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

B. Mean Squared Error (MSE): Average of squared differences between predicted and actual values.

$$\text{MSE} = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$$

C. Root Mean Squared Error (RMSE): Square root of MSE; penalizes larger errors more.

$$\text{RMSE} = \sqrt{MSE}$$

D. R^2 Score (Coefficient of Determination): Measures how well the regression model fits the data

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Ranges from 0 to 1 (higher is better);

3. Clustering Metrics

❖Used for unsupervised learning.

a. Silhouette Score

- Measures how similar an object is to its own cluster vs other clusters.

b. Davies-Bouldin Index

- Lower values indicate better clustering.

c. Adjusted Rand Index (ARI)

- Compares clustering results with ground truth labels (if available).